



Inspiring Excellence

Department of Computer Science and Engineering

Course: CSE447: Cryptography and Cryptanalysis

Semester: Summer 2026

Project Report

Title: Digital Land Record System

Submitted To: Sajid Imam Mahir, Farhan Faruk

Group No: 01

Section: 03

Submission Date: 06.09.2026

Group Members

No.	Full Name	Student ID
1	Maimuna Chowdhury	23241012
2	Danial Rahman	23201321
3	Golam Rabbani Shanto	22299527

Table of Contents

1. Introduction and System Overview	3
2. Login and Registration Module	4
3. User Data Encryption and Decryption	5
4. Password Hashing and Salting	6
5. Two-Factor Authentication (2FA)	7
6. Key Management Module	8
7. Post and Profile Management	9
8. Data Storage Security	10
9. Message Authentication Code (MAC)	11
10. Role-Based Access Control (RBAC)	12
11. Secure Session Management	13
12. GitHub Repository and Project Structure	14
13. Conclusion	15

1. Introduction and System Overview

This report documents the design, implementation, and security analysis of the CSE447 Lab Project. The system is a secure web application integrating multiple cryptographic protocols as required by the course specification. All encryption algorithms have been implemented from scratch without relying on built-in framework encryption functions.

1.1 Project Overview

A web application for transferring ownership of land deeds. It has two account types. Owners record which deeds they own and request transfers. Admins approve transfers. Every field is encrypted and stored in the database and decrypted when the user retrieves it.

[Briefly describe the web application — its purpose, users, and core functionality.]

1.2 Technology Stack

Backend: python, flask, jinja2 templates

Frontend: html, css, javascript

Database: SQLite 3

[List programming language, framework, database, and any external libraries used (excluding built-in crypto).]

1.3 System Architecture Diagram

2. Login and Registration Module

The system provides secure registration and login flows. New users supply credentials which are validated, encrypted, and persisted. During login, stored encrypted data is retrieved and decrypted for verification.

2.1 Registration Flow

A visitor supplies a username, an email address, and a password entered twice. The server validates that all fields are present, that the two passwords match, and that the password is at least eight characters, then performs registration as a single transaction:

1. Blind indexes are computed for the username and email — $\text{HMAC}(\text{value}, \text{BLIND_INDEX_KEY})$ over the lowercased, trimmed value — and checked for collisions, which is how uniqueness is enforced without storing either field in searchable plaintext.
2. An RSA keypair and an ECC keypair are generated for the new user and their private halves wrapped under the server master key.
3. The username, email and optional contact number are encrypted under the user's own new RSA public key.
4. The password is salted and hashed with PBKDF2, and only the digest, salt and iteration count are stored.
5. A messaging keypair is derived from the password (§7.1); only its public point and salt are recorded.
6. An HMAC tag is computed over the encrypted fields and the role, and stored with the row.

The ordering matters: both keypairs are generated before the user row is inserted, because the row's own fields are encrypted under the key that the row will then reference by version.

[Describe the step-by-step registration process (input validation → encryption → storage). Include a flowchart if possible.]

2.2 Login Flow

Login is two-step by construction, not by convention.

Step one looks the user up by username blind index and verifies the password with a constant-time comparison against the stored PBKDF2 digest. On success a session is created with `mfa_verified = 0`. This session is real — it exists in the database and the client holds its token — but every protected route validates with `require_mfa=True`, so it grants access to nothing.

Step two generates a six-digit code, stores only its HMAC, and emails the plaintext code to the address decrypted from the user's own record. When the user submits a matching code, the pending session is revoked and replaced rather than upgraded: a fresh token is issued and the old one is marked revoked in the same operation. A token observed before the second factor therefore can never become an authenticated session, which closes the session-fixation path that an in-place `mfa_verified = 1` update would leave open.

[Describe the login process (credential retrieval → decryption → comparison → session creation).]

2.3 Implementation Details

Requirement	Implementation Details
Login Module	<i>Username resolved by blind index (HMAC under BLIND_INDEX_KEY); password verified against a from-scratch PBKDF2-HMAC-SHA256 digest using a constant-time comparison. Success issues an unverified session that grants no access until the emailed one-time code is accepted, at which point the pending token is revoked and a fresh token issued. app/services/auth_service.py</i>
Registration Module	<i>Username, email, password entered twice (minimum eight characters), and an optional contact number. Uniqueness is enforced on username and email blind indexes rather than on plaintext. Registration issues an RSA keypair, an ECC keypair and a password-derived messaging key, encrypts all personal fields under the user's own RSA public key, and stores an HMAC tag over the row. app/services/user_service.py</i>
Data Encrypted Before Storage	<i>RSA-OAEP: username, email, contact; the four profile fields; every private key, wrapped under the server master key; and rejection reasons. EC-ElGamal over secp256k1: plot number, district, area and deed content; and transfer messages, under password-derived keys. Passwords are hashed with salted PBKDF2, not encrypted.</i>
Data Decrypted on Retrieval	<i>Each encrypted row records the key_version that wrote it. Reads call key_manager.get_key_version with that recorded version, never get_active_key, so records written under a since-rotated key remain readable. The HMAC tag is verified before any plaintext is returned; a failed tag raises rather than serving the record.</i>

Registration and login live in app/services/user_service.py and app/services/auth_service.py; the HTTP layer is app/routes/auth_routes.py. A failed login returns a single message, "Invalid username or password", whether the username is unknown or the password wrong, so the response does not disclose which accounts exist. The pending session cookie lasts 300 seconds; a verified session lasts 30 minutes.

3. User Data Encryption and Decryption

All sensitive user information (e.g., username, email, contact info) is encrypted before storage using asymmetric encryption algorithms implemented from scratch, and decrypted upon retrieval.

3.1 Fields Encrypted

RSA-OAEP: username, email and contact (users); full_name, address, phone and nid (profiles); every private key, wrapped under the server master key (user_keys); and an administrator's rejection reason (transfer_requests).

EC-ElGamal over secp256k1: plot_no, district, area and content (deeds); and the end-to-end transfer message, stored as two copies, one under each party's key (transfer_requests).

Stored unencrypted by design: password_hash is a salted PBKDF2 digest, not ciphertext; sessions.token_hash is a SHA-256 hash of a token that is itself never stored; public keys are public, and are HMAC-tagged instead; and username_idx, email_idx and plot_no_idx are blind indexes, irreversible HMACs that restore exact-match lookup over fields that are otherwise unsearchable.

[List all database fields that are encrypted (e.g., name, email, phone). Specify which algorithm is applied to each.]

3.2 Encryption Algorithm - RSA Implementation

app/crypto/rsa_service.py. Key generation picks two distinct random primes p and q of half the modulus size, forms $n = pq$ and $\phi = (p-1)(q-1)$, and derives d as the inverse of e modulo ϕ , retrying with fresh primes if $\gcd(e, \phi)$ is not 1. The public exponent is 65537 and the modulus is 1024 bits. Encryption is $c = m^e \bmod n$ and decryption $m = c^d \bmod n$, applied blockwise with a four-byte length prefix so that data longer than the modulus can be handled and its exact length recovered.

Modular exponentiation uses Python's built-in `pow(base, exponent, modulus)`, and the modular inverse `pow(e, -1, phi)`. These are general-purpose integer arithmetic, equivalent to the square-and-multiply and extended-Euclid routines they replace; key construction, encryption and decryption are implemented here. `sympy.randprime` supplies primes only.

Padding is EME-OAEP (RFC 8017), implemented in this file. It matters cryptographically rather than cosmetically: unpadded textbook RSA is deterministic, so an attacker holding the database could encrypt guesses under the stored public key and compare the results against stored ciphertext, recovering a low-entropy field such as an email address without ever touching a private key. OAEP mixes a fresh 32-byte random seed into every block, so the same plaintext encrypts differently each time. $DB = \text{SHA256}(\text{label}) \parallel \text{zero padding} \parallel 0x01 \parallel \text{message}$; $\text{maskedDB} = DB \oplus \text{MGF1}(\text{seed})$; $\text{maskedSeed} = \text{seed} \oplus \text{MGF1}(\text{maskedDB})$; $EM = 0x00 \parallel \text{maskedSeed} \parallel \text{maskedDB}$. Decoding reverses the two masks, verifies the label hash, and skips the padding to the 0x01 separator, raising on any structural failure rather than returning attacker-influenced bytes.

MGF1 and the label hash are built on our own SHA-256, so no library primitive appears at any point in the construction. The padding costs capacity: a k -byte modulus carries at most $k - 66$ bytes per block, or 62 bytes at 1024 bits, which block chunking hides from callers. The 1024-bit modulus is below modern practice and is a stated tradeoff, since pure-Python key generation at 2048 bits is too slow for an interactive demonstration.

[Explain your from-scratch RSA implementation: key size, padding scheme, modular exponentiation approach, etc.]

3.3 Encryption Algorithm - ECC Implementation

app/crypto/ecc_service.py. The curve is secp256k1: $y^2 = x^3 + 7$ over the prime field F_p , using the standard published p , base point G and order n . Point addition and doubling follow the chord-and-tangent formulas, with the point at infinity represented as `None`, and scalar multiplication uses double-and-add.

ECC is a family of curve arithmetic with no native encrypt operation, so an encryption scheme is required. The system uses EC-ElGamal: the private key is a scalar d and the public key is the point $Q = dG$; encryption draws a fresh ephemeral scalar k and outputs $C1 = kG$ and $C2 = M + kQ$; decryption recovers $M = C2 - dC1$.

Plaintext is mapped onto curve points by Koblitz encoding. A 30-byte chunk is read as an integer m , and candidate x -coordinates $x = 256m + \text{offset}$ are searched until one yields a quadratic residue, taking the square root by the shortcut available because p is congruent to 3 modulo 4; decoding discards the offset by integer division. A fresh ephemeral scalar is drawn for every chunk rather than once per message, so repeated chunks produce unrelated ciphertexts. Each chunk expands to 128 bytes, being two points of 64 bytes each.

ECIES was considered and rejected. It is the more standard elliptic-curve encryption scheme, but its payload layer requires a symmetric cipher, which the specification prohibits. EC-ElGamal is asymmetric throughout and is therefore the correct choice under this constraint, even though it is less compact.

[Explain your from-scratch ECC implementation: curve parameters (e.g., NIST P-256), point multiplication, encryption scheme (e.g., ECIES).]

3.4 How Both Algorithms Are Used Differently

The two algorithms hold disjoint responsibilities. Neither could be removed without leaving part of the system unencrypted.

RSA-OAEP protects identity and structure: account fields, profile fields, rejection reasons, and the wrapping of every user private key under the server master keypair. These are short, sparse fields, written once and read often, which suits RSA's block structure. ECC protects record content: the four deed fields and the end-to-end transfer messages. These are free text of unbounded length, where EC-ElGamal's 30-byte chunking is a natural fit and its per-chunk randomisation is most valuable.

The separation is enforced by the key manager rather than by convention: each user holds one RSA keypair and one ECC keypair, versioned and rotatable independently. Rotating a user's ECC key changes nothing about their account fields, and rotating their RSA key changes nothing about their deeds. Both behaviours are covered by tests.

[Clarify which part of the system uses RSA and which part uses ECC, satisfying the requirement that a single algorithm is not used for all operations.]

4. Password Hashing and Salting

Passwords are never stored in plaintext. A cryptographic hash function combined with a random salt is applied before storage to prevent dictionary and rainbow-table attacks.

4.1 Hashing Algorithm Used

Passwords are protected with PBKDF2 (RFC 2898) over HMAC-SHA256, with both HMAC-SHA256 (RFC 2104) and SHA-256 (FIPS 180-4) implemented from scratch in `app/crypto/`. SHA-256 is verified against the FIPS 180-4 test vectors and HMAC against RFC 4231, so correctness is demonstrated rather than assumed.

A plain hash is the wrong tool for a password, because SHA-256 is designed to be fast and speed is precisely what an offline attacker wants. PBKDF2 turns a fast hash into a deliberately slow key-derivation function by iterating it, so the defender pays the cost once per login while the attacker pays it once per guess. The derived key is 32 bytes, one SHA-256 block, so no outer PBKDF2 block loop is needed. The iteration count is 1000, which is a documented compromise: measured on this implementation a single HMAC-SHA256 call takes roughly 840 microseconds, so the recommended 600,000 iterations would add more than eight minutes to every login. The correct fix is a compiled primitive, not a smaller number.

[State the hash function used (e.g., SHA-256, SHA-3) and justify the choice.]

4.2 Salt Generation

Each password receives a 16-byte salt from `os.urandom`, stored as hexadecimal in `users.password_salt` alongside the digest and the iteration count.

Storing the salt beside the hash is correct, because a salt is not a secret. Its purpose is to make each digest unique, so that two accounts sharing a password produce different stored values and a precomputed rainbow table is useless against either. Storing the iteration count per row allows the cost to be raised later without invalidating existing accounts.

[Describe how the random salt is generated, its length, and where it is stored relative to the hash.]

4.3 Verification Process

`kdf.verify_password` re-derives the digest from the submitted password using the stored salt and iteration count, then compares it against the stored value.

The comparison XOR-accumulates every byte and tests the accumulator once at the end; it never returns early on a mismatch. An early-exit comparison leaks, through response timing, how many leading bytes were correct, which is enough to reconstruct a digest byte by byte. The same accumulate-then-test pattern is used for every tag comparison in the system.

[Explain how a password is verified on login (retrieve salt → re-hash input → compare).]

5. Two-Factor Authentication (2FA)

The system enforces two-step verification: the user must pass both primary credential validation and a second authentication factor before a session is granted.

5.1 2FA Method

The second factor is a six-digit one-time code delivered by email. After the password check succeeds the system generates a code, stores only its HMAC, decrypts the user's registered address from their own record, and sends the code over SMTP with STARTTLS. Only when a matching code is submitted is an authenticated session issued.

Four properties make the code more than a formality. Only HMAC(code) is stored, under a dedicated OTP_KEY held outside the database, so the code exists in the database in no form. The code is bound to one pending session by session_id rather than merely to the user, so a code issued for one sign-in attempt cannot complete a different one; an attacker who starts their own login against a victim's account therefore cannot have the victim read out "their" code. It is single-use and capped at three attempts, bounding online guessing against a space of 10^6 , and requesting a new code consumes the previous one. It expires after 300 seconds. Codes are generated by rejection sampling rather than by reducing random bytes modulo 10^6 , which would bias the distribution toward low codes.

This replaced an earlier TOTP implementation, and the change removed a weakness rather than merely altering the delivery channel. A TOTP secret is a long-lived shared secret held by both the server and the user, so a database compromise yields the ability to generate valid codes indefinitely; with an emailed code there is no long-lived per-user secret to steal, and the worst an attacker recovers is the HMAC of a single code that expires in five minutes. The residual limitation is inherent to the method, in that the code's confidentiality depends on the user's email account, which is exactly why it is short-lived, single-use and session-bound. STARTTLS protects the message in transit using the platform's TLS stack and performs none of this project's cryptography.

[Describe the second factor used (e.g., TOTP, email OTP, SMS code, security question). Explain the flow.]

5.2 Code Snippet

app/services/otp_service.py

```
def generate_code() -> str:
    # rejection sampling, so every code is equally likely;
    # random bytes modulo 10**6 would bias the low codes
    return str(random_below(10 ** DIGITS)).zfill(DIGITS)

def issue(conn, user_id, session_id) -> str:
    conn.execute("UPDATE login_otps SET consumed = 1 WHERE session_id = ?",
                  (session_id,))
    code = generate_code()
    issued = _now()
    conn.execute(
        "INSERT INTO login_otps (otp_id, user_id, session_id, code_hmac,"
        " attempts, consumed, issued_at, expires_at)"
        " VALUES (?, ?, ?, ?, 0, 0, ?, ?)",
        (str(uuid.uuid4()), user_id, session_id,
```

```

        hmac_hex(OTP_KEY, code.encode()),    # the code itself is never stored
        issued.isoformat(),
        (issued + timedelta(seconds=OTP_TTL_SECONDS)).isoformat()))
conn.commit()
return code

```

```

def verify(conn, session_id, code) -> bool:
    row = conn.execute(
        "SELECT * FROM login_otps WHERE session_id = ? AND consumed = 0"
        " ORDER BY issued_at DESC LIMIT 1", (session_id,)).fetchone()
    if row is None:
        return False
    if datetime.fromisoformat(row["expires_at"]) <= _now():
        _consume(conn, row["otp_id"]); return False
    if row["attempts"] >= OTP_MAX_ATTEMPTS:
        _consume(conn, row["otp_id"]); return False
    matched = bool(code) and verify_hmac(
        OTP_KEY, code.encode(), row["code_hmac"]) # constant time
    conn.execute("UPDATE login_otps SET attempts = attempts + 1,"
        " consumed = ? WHERE otp_id = ?",
        (1 if matched else 0, row["otp_id"]))
    conn.commit()
    return matched

```

app/services/auth_service.py, the two-step boundary

```

def complete_login(conn, token, code) -> str:
    # The pending token is revoked and replaced rather than upgraded, so a
    # token observed before the second factor can never become an
    # authenticated session.
    row = session_service.validate_session(conn, token, require_mfa=False)
    if row is None:
        raise AuthError("session is invalid or expired")
    if row["mfa_verified"]:
        raise AuthError("session already verified")
    if not otp_service.verify(conn, row["session_id"], code):
        raise AuthError("invalid or expired verification code")
    session_service.revoke_session(conn, token)
    return session_service.create_session(
        conn, row["user_id"], mfa_verified=True,
        ip_address=row["ip_address"], user_agent=row["user_agent"])

```

[Paste relevant 2FA implementation code here.]

6. Key Management Module

A dedicated Key Management Module handles the full lifecycle of cryptographic keys: secure storage, and rotation.

6.1 Key Storage Security

`app/crypto/key_manager.py` handles generation, distribution, storage, versioning and rotation for every key in the system.

Public keys are stored in plaintext, which is correct, since a public key is not secret. They are nonetheless HMAC-tagged, because substituting one would silently redirect every future encryption for that user to an attacker's keypair, and that substitution is undetectable by decryption alone: the victim's own reads keep working. Every private key is RSA-wrapped under a server master keypair held in `database/master_key.json`, outside the database and excluded from version control, so a stolen database file alone yields no usable private key. The tag over each key record covers the key id, user, algorithm, version, public key and wrapped private key together, so neither half can be swapped independently.

One key is deliberately not stored at all. The messaging key used for end-to-end transfer messages is derived from the user's password on demand and discarded after use. It is the single exception to this module's ownership of key material, and it is what makes the end-to-end claim honest: the master key can unwrap every key in `user_keys`, but it cannot produce a value that was never written down.

[Detail how private keys are protected in the database or file system (e.g., encrypted with another key, hardware token, etc.).]

6.2 Key Rotation Policy

Rotation is user-initiated from the Keys page rather than scheduled. In this system rotation is a response to a suspected compromise, and a fixed calendar interval would offer false assurance without addressing that event.

`rotate_key` marks the current version inactive, stamps `retired_at`, and issues the next version. Retired keys are retained, never deleted, and this is what makes rotation safe: every encrypted row records the `key_version` that wrote it, and reads fetch that specific version through `get_key_version`, so records written under a retired key remain readable immediately after rotation with no migration step. A separate re-encrypt action then walks the user's account, deeds and profile forward onto the new version at a time of their choosing. Rotation is also treated as a session event: all of the user's sessions are revoked, and the session performing the rotation is immediately re-issued, which signs out every other device without interrupting the operation in progress.

Adding the rotation interface exposed a bug that ordinary use could not. Three call sites decrypted stored records with the active key rather than the version that wrote them. On the dashboard this surfaced as `OverflowError`; in the authentication path it was silent, and locked rotated users out of their own accounts with no error at the point of failure. Every read of a stored record now goes through `get_key_version` with the version recorded on that row, and three regression tests hold the property in place.

[Describe the rotation schedule and how old keys are safely invalidated without breaking existing encrypted records.]

7. Post and Profile Management

Users can create, view, and edit posts, as well as view and update their profiles. All post and profile data is automatically encrypted before storage and decrypted on retrieval.

7.1 Post Module

Deeds are this system's posts. A deed carries a plot number, district, area and free-text content, and is owned by exactly one user. All four fields are encrypted with EC-ElGamal under the owner's ECC key.

Create fetches the owner's active ECC public key, encrypts the four fields, computes a blind index of the plot number for uniqueness and lookup, computes an HMAC tag over the four ciphertexts and the owner id, and records the key_version used. Read verifies the tag first: a mismatch raises rather than returning the record, and the interface renders a tampered state instead of content. Update is restricted to the owner in the service layer rather than in the template, and the plot number is immutable, because it is the record's real-world identifier and permitting edits would allow one parcel's record to be quietly relabelled as another's.

Transfer is requested by the owner and decided by an administrator. On approval the deed is re-encrypted under the new owner's ECC key, so the previous owner's key can no longer read it: ownership transfer is a cryptographic fact rather than a changed column value. Each decision carries an HMAC over its own fields, so an approval cannot be forged or altered directly in the database.

A requester may attach an end-to-end encrypted note. Each user holds a third keypair whose private scalar is derived from their password, as $d = \text{PBKDF2}(\text{password}, \text{label} || \text{salt})$ reduced modulo the curve order, with public point $Q = dG$, and this scalar is never stored. The database holds Q and the salt only, so an adversary holding the entire database and the master key file still cannot read a message. The note is encrypted with EC-ElGamal under the recipient's Q , with a second copy under the sender's own; reading either requires re-entering the password, and the scalar is discarded immediately after use. The salt and the domain-separation label differ from those used for password hashing, which is load-bearing: were they identical, the derived scalar would equal the stored password digest and the private key would sit in the database in plaintext. Because EC-ElGamal carries no integrity of its own, and decryption with a wrong scalar returns plausible bytes rather than failing, a four-byte canary is prepended before encryption and checked afterwards, so an incorrect password is rejected cleanly instead of rendering as noise.

[Describe the Create / Read / Update flow for posts. List all fields that are encrypted and the algorithm applied.]

7.2 Profile Module

A profile holds four optional fields, being full name, address, phone number and national ID number, each encrypted under the user's RSA public key, in contrast to the ECC used for deeds. Profiles are one-to-one with users and are created lazily on first update; requesting a profile that does not yet exist returns empty fields rather than an error.

Update accepts a subset of fields, rejects any field name outside the permitted set, re-encrypts under the user's active RSA key, recomputes the HMAC tag over all four ciphertexts and the user id, and records the new key_version.

Retrieval verifies the tag before decrypting and raises IntegrityFailure if it does not match, which the interface renders as a tampered-profile warning rather than showing partial data.

[Describe the profile view / update flow. List which profile fields are encrypted.]

7.3 Screenshots

post creation:

DLR

Digital Land
Record System

Dashboard

Deeds

Profile

Transfers

Keys

OWNER

Sign out

New deed

Plot number

15

District

Bunyland

Area (acres)

2

Deed content

Owned by cat

Create deed

[Back to deeds](#)

post listing:

DLR

Digital Land
Record System

Dashboard

Deeds

Profile

Transfers

Keys

OWNER

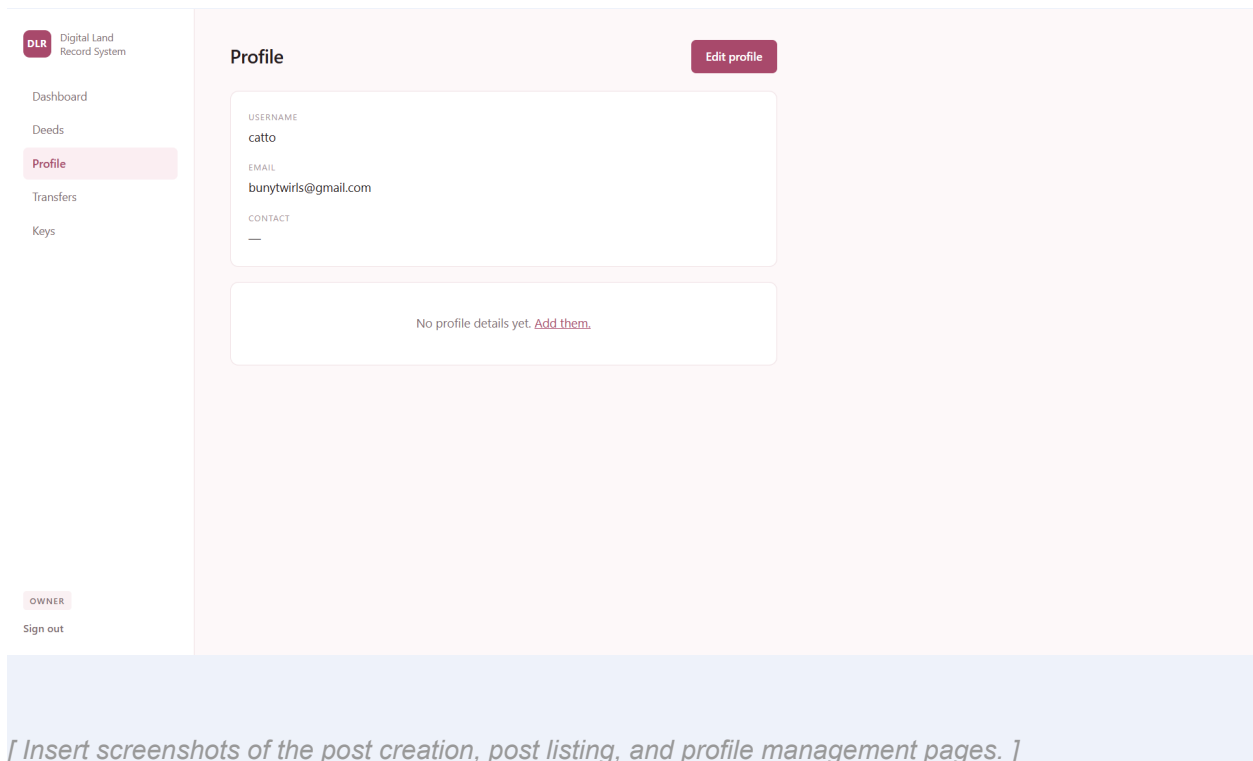
Sign out

Deeds

New deed

PLOT	DISTRICT	KEY	UPDATED
20	Bunyland	v1	2026-09-06 17:22

profile management:



8. Data Storage Security

All critical data - user information, posts, and cryptographic keys - is stored in encrypted form to prevent plaintext access even in the event of a database compromise.

8.1 Evidence of Encrypted Storage

Every field of consequence is stored as base64 ciphertext in a TEXT column. The database contains no plaintext username, email, contact number, deed content, profile field or transfer message, and no plaintext private key. This is checked continuously rather than only by inspection: the test suite asserts that registered personal data, deed content, wrapped private keys, one-time codes, transfer messages and the derived messaging scalar each fail to appear anywhere in a dump of the database. Two categories are stored unencrypted by design. Public keys are public, and are HMAC-tagged against substitution. Blind indexes are irreversible HMACs rather than ciphertext: encrypted values cannot be searched, and must not be, since the encrypting key may rotate, so storing HMAC(value) alongside restores exact-match lookup without storing the plaintext or any reversible transformation of it. The residual leak is recorded honestly: ciphertext length is a function of plaintext length, so an observer learns approximately how long a field is, though not its content.

```
33  CREATE TABLE profiles (
42  updated_at TEXT NOT NULL
43  )//
44  user_id TEXT PRIMARY KEY,
45  username_idx TEXT NOT NULL UNIQUE, -- blind index: HMAC(username, lookup_key)
46  username_enc TEXT NOT NULL, -- RSA
47  email_idx TEXT NOT NULL UNIQUE, -- blind index
48  email_enc TEXT NOT NULL, -- RSA
49  contact_enc TEXT, -- RSA
50  password_hash TEXT NOT NULL, -- your PBKDF2-style KDF
51  password_salt TEXT NOT NULL,
52  kdf_iterations INTEGER NOT NULL,
53  role TEXT NOT NULL CHECK (role IN ('ADMIN','OWNER')),
54  key_version INTEGER NOT NULL, -- which RSA key encrypted the above
55  hmac_tag TEXT NOT NULL, -- integrity over the encrypted fields
56  is_active INTEGER NOT NULL DEFAULT 1,
57  created_at TEXT NOT NULL
58  ))
59  //
60  %NUL
61  %NUL
62  %NUL
63  %NUL
64  Me6e9ea5d-a5a5-4fd7-8010-c7b3b18f20abeb852867cab709895f7fc4328f03de840cda049ed33d7f33a6e4214284fb0151OoyVz6baKQ
65  eb852867cab709895f7fc4328f03de840cda049ed33d7f33a6e4214284fb0151
66  d64cc6b2d404ad7660fab79623267124ba7179d1670e8445fbc04620a64b32cf
```

[Insert a screenshot of the raw database records (showing ciphertext, not plaintext) to demonstrate encrypted storage.]

9. Message Authentication Code (MAC)

Message Authentication Codes (MACs) are used to verify the integrity of stored data and detect any unauthorized modifications. The system implements CBC-MAC or HMAC as required.

9.1 MAC Algorithm Used

The system uses HMAC-SHA256 (RFC 2104), implemented from scratch in `app/crypto/hmac_service.py` on top of the project's own SHA-256, and verified against the RFC 4231 test vectors. The construction is the standard one, $H((K \text{ XOR } \text{opad}) \parallel H((K \text{ XOR } \text{ipad}) \parallel m))$, with keys longer than the 64-byte block hashed down and shorter keys zero-padded.

CBC-MAC was rejected on the specification's own terms. CBC-MAC requires a block cipher, and a block cipher is symmetric encryption, which the project prohibits; HMAC needs only a hash function, so it is the only one of the two offered options that can be built within the constraint. It is also the stronger choice independently, since raw CBC-MAC is insecure for variable-length messages without additional construction, and the records here vary in length.

Verification XOR-accumulates over the full tag and tests once, never exiting early.

[State whether CBC-MAC or HMAC is used. Justify the choice. Describe the implementation from scratch.]

9.2 Integrity Verification Flow

Encryption hides a record's contents but does nothing to prevent someone with database access from altering or replacing ciphertext. A MAC keyed with a secret held outside the database, `INTEGRITY_KEY` in `.env`, makes any such change detectable. Tags are computed on every write and verified on every read, not merely on privileged operations.

Tags cover the encrypted username, email, contact and role on a user row; all four encrypted fields and the user id on a profile; the four encrypted fields and the owner id on a deed; the key id, user, algorithm, version, public key and wrapped private key on a key record; the request, deed, both parties, status, timestamp and deciding administrator on a transfer decision; and the request id together with both ciphertexts on a message. Including the role is what makes privilege escalation by direct database edit detectable, since changing `OWNER` to `ADMIN` invalidates the tag. Including the deciding administrator and the timestamp means a rejected transfer cannot be rewritten as approved. On failure the system raises or flags rather than serving the record: the deed and profile views render explicit integrity-check warnings, and the key manager raises `KeyIntegrityError`. An `audit_logs` table with a hash-chain design exists in the schema but is not yet populated, and is listed here as designed rather than claimed as implemented.

[Describe when and how MAC verification is performed (on every read, on critical operations, etc.).]

10. Role-Based Access Control (RBAC)

Role-Based Access Control defines distinct privilege levels for Administrators and Regular Users, ensuring that sensitive operations are restricted appropriately.

10.1 Roles Defined

There are two roles, constrained at the database level by CHECK (role IN ('ADMIN','OWNER')). An OWNER is a landowner: they register, authenticate, record deeds, read and edit only their own, maintain their own profile, request transfers of deeds they own, read messages addressed to them, and manage and rotate their own keys. An ADMIN holds every OWNER capability over their own records and additionally reviews the transfer queue and approves or rejects requests.

An administrator may view any deed, because deciding a transfer requires seeing what is being transferred, but may not edit one: adjudication and modification are deliberately separate powers, enforced in deed_service rather than in the interface. Roles are assigned at registration and changed only through the make_admin.py maintenance script, so no request can grant privilege.

Enforcement lives in app/routes/deps.py, whose login_required and admin_required decorators run before any view body executes, while ownership checks live in the service layer so that they hold regardless of which route reaches them.

[List all roles in the system (e.g., Admin, User) and their high-level responsibilities.]

10.2 Permission Matrix

Operation / Resource	Admin	Regular User
View own profile	Yes	Yes
Edit own profile	Yes	Yes
Create / Edit posts	Create: yes. Edit: own deeds only	Create: yes. Edit: own deeds only
Delete any post	No - deletion is not implemented; deeds are permanent records	No - deletion is not implemented
View all user accounts	No - no such view exists; the public key directory is visible to all	No
Manage / rotate keys	Own keys only	Own keys only
Assign roles to users	No - offline make_admin.py script only	No
View audit logs	Not implemented - the audit_logs table is designed but unpopulated	Not implemented

(Add or remove rows as appropriate for your system. Tick marks above are placeholders.)

11. Secure Session Management

Authentication tokens and session identifiers are managed securely to prevent session hijacking, fixation, and replay attacks.

11.1 Token Signing / Verification

Session tokens are random, not signed. `create_session` generates 32 bytes from `os.urandom`, returns the hexadecimal token to the client once, and stores only its SHA-256 hash; verification hashes the presented token and looks up the hash.

The choice is deliberate. A signed token such as a JWT carries its claims in the client's hands and remains valid until it expires, so revoking one early requires a server-side denylist, which is a session table under a different name. An opaque random token is a pointer into server-side state, so revocation is a single row update and takes effect immediately; and because only the hash is stored, a stolen database cannot be replayed against a live session, which is the same reasoning that governs password storage applied to tokens.

Each session row carries the issue time, a 30-minute expiry, the client IP address and user agent, an `mfa_verified` flag and a `revoked` flag, and `validate_session` rejects a token that is unknown, revoked, past expiry, or not yet verified. Session fixation is addressed by revoking the pending session and issuing a fresh token on successful verification rather than upgrading it in place. Cookies are set `HttpOnly`, placing them out of reach of JavaScript, and `SameSite=Lax`, so they are not attached to cross-site requests. Rotating a key revokes every session for that user, and `logout` revokes rather than deletes so the record survives for audit. Two limitations are stated rather than glossed: the `Secure` flag is false because the demonstration runs over plain HTTP and must be true behind TLS in deployment, and there is no CSRF token on forms, which is outside the required feature set but a genuine gap in a production deployment.

[Describe how tokens are signed (e.g., HMAC-SHA256 or RSA) and how they are verified on each request.]

12. GitHub Repository and Project Structure

Field	Details
GitHub Repository URL	https://github.com/DigitalLandRecordSystem/digital-land-record-system.git

12.1 Repository Structure

```
digital-land-record-system/  
|-- backend/  
| |-- app/  
| | |-- crypto/          from-scratch primitives  
| | | |-- hashing.py     SHA-256 (FIPS 180-4)  
| | | |-- hmac_service.py HMAC-SHA256 (RFC 2104)  
| | | |-- kdf.py         PBKDF2 (RFC 2898)  
| | | |-- rsa_service.py  RSA  
| | | |-- ecc_service.py  ECC  
| | | |-- key_manager.py  generation, wrapping, versioning, rotation  
| | | |-- messaging_key.py password-derived key, never stored  
| | | |-- blind_index.py  searchable lookup tokens  
| | | |-- integrity.py    record tags  
| | | +-- random_gen.py   CSPRNG helpers, rejection sampling  
| | |-- services/        business logic  
| | | |-- user_service.py, auth_service.py, otp_service.py, mailer.py  
| | | |-- session_service.py, deed_service.py, profile_service.py  
| | | +-- transfer_service.py, message_service.py  
| | |-- routes/          Flask blueprints  
| | | |-- __init__.py     create_app  
| | | |-- deps.py         RBAC decorators, request-scoped connection  
| | | +-- auth_, main_, deed_, profile_, transfer_, key_routes.py  
| | |-- database/db.py    connection handling  
| | +-- config.py         .env loading and secret validation  
| |-- tests/             113 tests  
| |-- run.py             development entry point  
| |-- init_setup.py       creates the database and server master key  
| |-- make_admin.py       promotes a user to ADMIN  
| |-- requirements.txt     pinned  
| +-- .env.example  
|-- database/  
| |-- schema.sql  
| |-- land_records.db      (gitignored)  
| +-- master_key.json      (gitignored)  
|-- frontend/  
| |-- templates/          Jinja templates  
| +-- static/style.css
```

```
|-- docs/  
+-- README.md
```

Test coverage is organised primitive by primitive. `test_hashing.py` and `test_hmac_service.py` run the published FIPS 180-4 and RFC 4231 vectors; `test_rsa_service.py` and `test_ecc_service.py` cover round-trips, boundary lengths, randomisation and wrong-key behaviour; `test_key_manager.py` covers wrapping and rotation; and the service-level files cover access control, integrity detection, behaviour across key rotation, two-step login and end-to-end messaging.

[Paste or describe the top-level folder/file structure of your repository (e.g., output of `tree` command or a directory listing).]

12.2 README Overview

The README documents a requirements-to-code map showing where each specification item is implemented, the cryptographic design and its rationale, and the project structure.

Setup instructions cover creating a virtual environment, installing the pinned dependencies from `requirements.txt`, copying `.env.example` to `.env` and generating the secret keys, and creating the database and server master key with `init_setup.py`. Running instructions cover the development server, started with `python run.py` and served at `127.0.0.1:5000`, and the test suite, run with `python -m pytest tests/ -v`. Python 3.11 or newer is required.

It also lists the known limitations, each stated as a considered tradeoff rather than an oversight, and notes that `.env` and `database/master_key.json` hold key material, are excluded from version control, and that disclosure of either is a key-rotation event.

[Summarise what your README.md contains: setup instructions, environment requirements, how to run the project locally.]

13. Conclusion

The system meets each requirement in the specification, and does so with every cryptographic primitive written from first principles: SHA-256, HMAC, PBKDF2, RSA with OAEP padding, and elliptic-curve arithmetic with EC-ElGamal encryption. RSA and ECC hold genuinely separate responsibilities rather than being used interchangeably to satisfy a rule, and a suite of 113 tests holds these properties in place. Several of those tests exist because a property was found to be broken.

The most instructive difficulties were the ones where a design that looked finished turned out not to be. Rotation exposed a bug that ordinary use could not: three call sites decrypted stored records with the active key rather than the version that wrote them, which on the dashboard raised an overflow error and in the authentication path failed silently, locking rotated users out of their own accounts. Features whose absence hides defects are worth building early.

Constraints also ruled out the standard answers. ECIES is the conventional elliptic-curve encryption scheme and CBC-MAC an explicitly offered MAC, and both were unusable here for the same reason, in that each requires a symmetric primitive. Working within that constraint led to EC-ElGamal with Koblitz encoding and to HMAC, and understanding why those were the available choices proved more valuable than reaching for the familiar ones. Small parameter choices carried real weight too: a public exponent of 11 was replaced with 65537 after it emerged that short messages satisfied $m^e < n$, so no modular reduction occurred and any private key recovered them, and adding OAEP padding closed a live attack rather than tidying the implementation.

The most useful single idea was the distinction between data that is encrypted and data that is encrypted such that the operator cannot read it. An earlier plan to encrypt transfer messages under each user's stored ECC key would have produced something encrypted at rest but not end-to-end, since the server master key can unwrap any stored private key; deriving a separate messaging key from the user's password, and storing it nowhere, was what made the claim true. The result is not production-grade, and the report is explicit about where and why: 1024-bit RSA moduli, 1000 KDF iterations, no CSRF protection, cookies without the Secure flag, ciphertext that leaks plaintext length, and an audit-log table designed but unpopulated. Each is a stated tradeoff between the cost of pure-Python cryptography and the need for an interactive demonstration.

[Write a brief concluding paragraph reflecting on challenges faced, lessons learned, and the overall outcome of the project.]